

ASSIGNMENT-2

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML)?

Ans:

Large Language Models (LLMs) – A

I rate myself **A in LLMs** because I can independently design and implement LLM-based applications. In my Kerala Ayurveda Chatbot project, I built an end-to-end Retrieval-Augmented Generation (RAG) pipeline, covering document ingestion, text chunking, embedding generation, vector-based retrieval, prompt construction, and response generation using an LLM. I am comfortable coding, debugging, and improving such systems without supervision.

Deep Learning – A

I rate myself **A in Deep Learning** based on my hands-on experience building a **weed detection system** using deep learning techniques. In that project, I worked with convolutional neural networks for image-based classification, handled data preprocessing and augmentation, and understood how model architecture, loss functions, and training dynamics impact performance. This experience gives me a solid practical understanding of how deep learning models are trained and applied in real-world scenarios.

Machine Learning – B

I rate myself **B in Machine Learning**. I understand core ML concepts such as feature representation, similarity metrics, model evaluation, and optimization, and I can implement ML components with guidance. In my projects, I have applied ML principles through embedding-based similarity search and retrieval pipelines, although my primary focus is more on applied GenAI and system-level AI development than classical ML model training.

Artificial Intelligence (Overall) – A

I rate myself **A in AI overall** because I can independently design and build complete AI systems that combine multiple AI paradigms. Along with LLM-based projects, I have worked on **agentic AI systems**, including a **research agent** for information gathering and synthesis and an **AI hiring assistant** for resume analysis and candidate-job matching. These projects demonstrate my ability to architect multi-component AI solutions involving reasoning, decision-making, and automation rather than isolated model

2. What are the key architectural components to create a chatbot based on LLM?.

Designing an LLM-based chatbot requires selecting the **right architecture based on the nature of data, accuracy requirements, scalability, latency, and cost**. Below is a **step-by-step architectural explanation**, with clarity on **when, where, and why** each component is used.

Step 1: Choosing the Overall Architecture (Normal LLM vs RAG vs Hybrid) Normal LLM (Without External Data)

When to use

- General conversational chatbots
- Questions that rely on common, public knowledge

Why

- Simple architecture
- Low latency
- No external data pipeline

Limitations

- Cannot handle private or domain-specific data
- High risk of hallucinations

Conclusion

Not suitable for domain-specific systems like knowledge-based chatbots.

Retrieval-Augmented Generation (RAG) – Chosen Approach

When to use

- Domain-specific knowledge (medical, legal, internal documents)
- Medium to large static datasets
- High accuracy requirements

Why

- Grounds LLM responses in real data
- Reduces hallucinations
- No need for expensive fine-tuning

Usage in my project

I selected **RAG** for the Kerala Ayurveda Chatbot because responses must be generated strictly from Ayurvedic source content rather than general LLM knowledge.

Advanced Architectures (Used Only When Required)

- **RAG + Re-ranking** → When precision is critical
- **Graph RAG** → When data has strong entity relationships
- **Hybrid Search** → When combining keyword and semantic search

For text-heavy Ayurvedic data, **standard RAG is sufficient and optimal.**

Step 2: Data Ingestion and Preprocessing

Why This Step Is Needed

LLMs cannot directly process raw documents. Data must be ingested, cleaned, and structured.

What is done

- Load domain documents
- Normalize text
- Attach metadata for filtering

When complexity increases

- Multiple document formats
 - Structured and unstructured data combined
-

Step 3: Text Chunking Strategy

Why Chunking Is Required

LLMs have context window limits, and embeddings work best on focused semantic units.

Large Chunk Size

Advantages

- More context per chunk
- Smaller vector index

Disadvantages

- Reduced retrieval precision
- Higher token usage and cost

When to use

- Narrative or descriptive documents
-

Small Chunk Size

Advantages

- High retrieval precision
- Better semantic matching

Disadvantages

- Loss of broader context
- Larger vector database

Used

For Q&A-based systems like Ayurveda consultation, **medium-to-small chunk sizes** provide the best balance between accuracy and context.

Step 4: Embedding Model Selection

Why Embeddings Are Needed

Embeddings convert text into numerical vectors that capture **semantic meaning**, enabling similarity search.

Embedding Choice Strategy

- **Open-source embeddings** → Cost-effective, good performance
- **API-based embeddings** → Higher quality, higher cost

Used in my project

I used **transformer-based open-source embeddings** to maintain cost efficiency while achieving sufficient semantic accuracy.

Step 5: Vector Database Selection

Why a Vector Database Is Required

Traditional databases cannot perform semantic similarity search efficiently.

Selection Criteria

- Dataset size
- Latency requirements
- Cost constraints

Scenario	Recommended Database	Reason
Local or demo projects	FAISS	Fast, lightweight, free
Enterprise scale	Pinecone	Fully managed, scalable
Hybrid search	Weaviate	Keyword + vector support
Rapid prototyping	ChromaDB	Simple integration

Used

I selected **FAISS** because it is efficient, cost-free, and ideal for internal or demo-based RAG systems.

Step 6: LLM Selection (Groq vs ChatGPT vs Open Source)

Selection Factors

- Latency
- Accuracy
- Cost
- Deployment requirements

Groq (Used in My Project)

- Very low latency
- Cost-efficient inference
- Suitable for real-time chatbots

ChatGPT (OpenAI APIs)

- Strong reasoning capability
- Higher operational cost

Open-Source LLMs

- Full control
- Infrastructure and maintenance overhead

Used

For my chatbot, **Groq-based LLMs** provide the best balance of **speed and cost efficiency**.

Step 7: Cost Optimization Considerations

Component	Cost Impact
Chunk size	Larger chunks increase token cost
Embeddings	Paid models add recurring cost
Vector DB	Managed services incur monthly charges
LLM	Advanced models increase inference cost

Cost Strategy Used

- Open-source embeddings
- FAISS for storage
- Low-latency Groq inference
- No fine-tuning overhead

Step 8: Evaluation and Validation

Why Evaluation Is Important

LLMs may produce fluent but incorrect answers.

Evaluation Techniques

- Manual domain validation
- Context-grounded answer checks
- Query coverage testing
- Latency monitoring
- Hallucination analysis

When Advanced Evaluation Is Needed

- Production systems
- Regulated domains

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why

A vector database is a specialized database used to store and search vector embeddings, which are numerical representations of data such as text, images, or audio. These embeddings capture the semantic meaning of the data rather than exact words.

Unlike traditional databases that work well with exact matches or structured queries, vector databases enable similarity-based search. This allows systems to retrieve relevant information even when the wording of a query is different from the stored content.

For example, queries like *“Ayurvedic remedies for digestion”* and *“Treatment for weak digestive fire”* may not share the same keywords, but a vector database can still recognize them as related because their embeddings are close in vector space.

Why Vector Databases Are Important

Vector databases are especially important in LLM-based applications, where:

- Users can phrase the same question in many different ways
- Data is unstructured (documents, PDFs, articles, FAQs)
- The system needs semantic understanding, not keyword matching

In Retrieval-Augmented Generation (RAG) systems, vector databases act as the knowledge layer, allowing the LLM to retrieve relevant context before generating a response. This significantly improves accuracy and reduces hallucinations.

When Should a Vector Database Be Used?

A vector database should be used when:

- The application requires semantic search
- Data is text-heavy and unstructured
- User queries are flexible and unpredictable
- An LLM needs grounding in external or private knowledge

It is not necessary when:

- Data is fully structured
 - Exact matches are sufficient
 - Semantic understanding is not required
-

EXAMPLE:

Problem:

Build a domain-specific question-answering chatbot for internal documents (e.g., healthcare guidelines, company policies, or educational content), where users can ask questions in natural language and expect accurate, context-based answers.

Constraints:

- Dataset size is moderate (thousands of documents)
 - Low latency is required
 - Cost needs to be controlled
 - The system is for internal or demo use
-

Vector Database Choice: FAISS

For this hypothetical problem, I would choose FAISS.

Why FAISS?

I would select FAISS because:

- It is lightweight and fast, suitable for moderate-sized datasets
- It is open-source and free, making it cost-effective
- It integrates easily with frameworks like LangChain
- It works well for local or internal deployments

FAISS is ideal when full control, low latency, and minimal operational overhead are important. For a proof-of-concept or internal knowledge system, it provides everything needed without introducing unnecessary complexity or cost.

When I Would Choose a Different Vector Database

- Pinecone → For large-scale, production-grade systems requiring high availability and managed infrastructure
- Weaviate → When hybrid search (keyword + vector) or metadata-heavy queries are required
- ChromaDB → For rapid prototyping and experimentation

The choice depends on scale, cost, and deployment needs rather than a one-size-fits-all solution.